MODULE 7 – SQL & DATABASE

1. Introduction to Database

A database is an organized collection of data that can be stored, accessed, managed and updated easily.

Example:

A college database may contain:

- Student details.
- Faculty details.
- Department details.
- Course details.
- Examination results.

Why do we use databases?

- To store large amounts of data.
- To retrieve data easily.
- To update and delete records.
- To maintain data accuracy.
- To avoid unnecessary duplication.
- To provide controlled access to data.
- To support multiple users and applications.

2. What is DBMS?

DBMS stands for Database Management System.

A DBMS is software used to create, store, manage and retrieve data from databases.

Examples:

- MySQL
- Microsoft SQL Server
- Oracle Database
- PostgreSQL
- SQLite

Advantages of DBMS:

1. Organizes data efficiently.
2. Reduces unnecessary data duplication.
3. Provides data security.
4. Supports backup and recovery.
5. Maintains data integrity.
6. Supports multiple users.
7. Provides controlled access to data.

3. What is RDBMS?

RDBMS stands for Relational Database Management System.

It stores data in tables containing rows and columns. Tables can be connected using relationships.

Examples:

- Microsoft SQL Server
- MySQL
- PostgreSQL
- Oracle Database

Example:

Students Table:

StudentId	Name	DepartmentId
1	Arun	101
2	Divya	102
3	Kumar	101

Departments Table:

DepartmentId	DepartmentName
101	CSE
102	ECE

The DepartmentId column connects the Students and Departments tables.

4. DBMS vs RDBMS

DBMS:

- General database management system.
- Different DBMS products may use different data models.
- Data organization depends on the database model.

RDBMS:

- Uses the relational model.
- Stores data in tables.
- Supports relationships between tables.
- Uses keys and constraints to maintain data integrity.
- Commonly uses SQL.

5. Introduction to SQL

SQL stands for Structured Query Language.

SQL is used to communicate with relational databases.

It allows developers to:

- Create databases and tables.
- Insert records.
- Retrieve records.
- Update records.
- Delete records.
- Manage database structures.
- Control database access.
- Manage transactions.

Example:

```
None
SELECT * FROM Students;
```

This query retrieves all columns and records from the Students table.

6. Introduction to Microsoft SQL Server

Microsoft SQL Server is a relational database management system developed by Microsoft.

It is widely used with:

- C# applications.
- ASP.NET Core Web APIs.
- MVC applications.
- Enterprise applications.
- Entity Framework Core.

Important SQL Server Tools:

1. SQL Server Management Studio (SSMS): Used to connect to SQL Server, write queries and manage databases.
2. SQL Server Express: A free edition of SQL Server suitable for development and smaller applications, subject to its edition limits.
3. SQL Server Database Engine: The service that stores and processes database information.
4. SQL Server Agent: Used to schedule jobs in editions that support it.

7. Database, Table, Row and Column

Database:

A collection of related data and database objects.

Table:

A structure used to store data in rows and columns.

Row:

A single record in a table.

Column:

A field that describes a particular property of the data.

Example:

StudentId	Name	Age
1	Arun	21
2	Divya	22

Here:

- Students is the table.
- StudentId, Name and Age are columns.
- Each student entry is a row.

8. SQL Data Types

Data types define what kind of values a column can store.

Numeric Data Types:

- INT: Integer values.
- BIGINT: Large integer values.
- SMALLINT: Small integer values.
- TINYINT: Small non-negative integer values in SQL Server.
- DECIMAL(p,s): Exact decimal values.
- NUMERIC(p,s): Equivalent to DECIMAL in SQL Server.
- FLOAT: Approximate floating-point values.
- REAL: Approximate floating-point values.

Character and String Data Types:

- CHAR(n): Fixed-length non-Unicode character data.
- VARCHAR(n): Variable-length non-Unicode character data.
- NCHAR(n): Fixed-length Unicode character data.
- NVARCHAR(n): Variable-length Unicode character data.
- VARCHAR(MAX): Large variable-length non-Unicode data.
- NVARCHAR(MAX): Large variable-length Unicode data.

Date and Time Data Types:

- DATE: Stores date.
- TIME: Stores time.
- DATETIME: Stores date and time.
- DATETIME2: Stores date and time with greater precision.
- DATETIMEOFFSET: Stores date, time and time-zone offset.

Other Data Types:

- BIT: Stores 0, 1 or NULL.
- UNIQUEIDENTIFIER: Stores a GUID.
- VARBINARY: Stores binary data.
- XML: Stores XML data.

Example:

```
None
CREATE TABLE Students
(
    StudentId INT,
    Name NVARCHAR(100),
    Age INT,
    Mark DECIMAL(5,2),
    IsActive BIT,
    AdmissionDate DATE
);
```

9. Categories of SQL Commands

SQL commands are commonly grouped into five categories.

1. DDL – Data Definition Language

Used to define or modify database structures.

Commands:

- CREATE
- ALTER
- DROP
- TRUNCATE

2. DML – Data Manipulation Language

Used to insert, update and delete data.

Commands:

- INSERT
- UPDATE
- DELETE

3. DQL – Data Query Language

Used to retrieve data.

Command:

- SELECT

4. DCL – Data Control Language

Used to manage database permissions.

Commands:

- GRANT
- REVOKE
- DENY (SQL Server)

5. TCL – Transaction Control Language

Used to control transactions.

Commands:

- BEGIN TRANSACTION
- COMMIT
- ROLLBACK
- SAVE TRANSACTION

10. Creating a Database

The CREATE DATABASE command creates a new database.

Syntax:

```
None
CREATE DATABASE DatabaseName;
```

Example:

```
None
CREATE DATABASE CollegeDB;
```

To use the database:

```
None
USE CollegeDB;
```

To view databases in SQL Server:

```
None
SELECT name
FROM sys.databases;
```

11. Creating a Table

The CREATE TABLE command creates a new table inside a database.

Syntax:

```
None
CREATE TABLE TableName
(
    Column1 DataType,
    Column2 DataType
);
```

Example:

```
None
CREATE TABLE Students
(
    StudentId INT,
    Name NVARCHAR(100),
    Age INT,
    Department NVARCHAR(50)
);
```

12. SQL Constraints

Constraints are rules applied to table columns to maintain data accuracy and integrity.

Important constraints:

1. PRIMARY KEY

Uniquely identifies each record in a table.

- Cannot contain duplicate values.

- Cannot contain NULL values.
- A table can have only one primary key constraint, which may contain multiple columns.

Example:

```
None
CREATE TABLE Students
(
    StudentId INT PRIMARY KEY,
    Name NVARCHAR(100)
);
```

2. FOREIGN KEY

Creates a relationship between two tables.

It ensures that a referencing value corresponds to a candidate key in the referenced table, subject to the constraint.

Example:

```
None
CREATE TABLE Departments
(
    DepartmentId INT PRIMARY KEY,
    DepartmentName NVARCHAR(100)
);

CREATE TABLE Students
(
    StudentId INT PRIMARY KEY,
    Name NVARCHAR(100),
    DepartmentId INT,
    FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

3. UNIQUE

Ensures that values in a column or combination of columns are unique.

Example:

```
None
CREATE TABLE Users
(
    UserId INT PRIMARY KEY,
    Email NVARCHAR(100) UNIQUE
);
```

4. NOT NULL

Prevents a column from storing NULL values.

Example:

```
None
CREATE TABLE Employees
(
    EmployeeId INT,
    Name NVARCHAR(100) NOT NULL
);
```

5. CHECK

Ensures that inserted or updated values satisfy a specified condition.

Example:

```
None
CREATE TABLE Students
(
    StudentId INT PRIMARY KEY,
    Age INT CHECK (Age >= 18)
);
```

6. DEFAULT

Provides a default value when no value is supplied for the column.

Example:

```
None
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Status NVARCHAR(20) DEFAULT 'Active'
);
```

13. Primary Key and Foreign Key

Primary Key:

A primary key uniquely identifies each record in a table.

Example:

Students:

StudentId	Name
1	Arun
2	Divya

StudentId is the primary key.

Foreign Key:

A foreign key references a primary key or another eligible unique key in a related table.

Example:

Departments:

DepartmentId	DepartmentName
101	CSE
102	ECE

Students:

StudentId	Name	DepartmentId

1	Arun	101
2	Divya	102

DepartmentId in Students is a foreign key referencing Departments.

Difference:

- Primary key identifies a record in its own table.
- Foreign key establishes a relationship with a referenced table.
- A table has one primary key constraint, but it can have multiple foreign key constraints.
- Foreign key values may repeat.
- A foreign key column can contain NULL when it is not defined as NOT NULL.

14. Other Types of Keys

Candidate Key:

A minimal set of columns that can uniquely identify a record.

Super Key:

Any set of one or more columns that uniquely identifies a record.

Alternate Key:

A candidate key that is not selected as the primary key.

Composite Key:

A key formed using two or more columns.

Example:

```
None
CREATE TABLE StudentCourses
(
    StudentId INT,
    CourseId INT,
    EnrollmentDate DATE,
    PRIMARY KEY (StudentId, CourseId)
);
```

Here, StudentId and CourseId together form a composite primary key.

Surrogate Key:

An artificial key generated mainly to identify records.

Example: An automatically generated StudentId.

15. INSERT Statement

The INSERT command is used to add new records to a table.

Insert One Record:

```
None
INSERT INTO Students
    (StudentId, Name, Age, Department)
VALUES
    (1, 'Arun', 21, 'CSE');
```

Insert Multiple Records:

```
None
INSERT INTO Students
    (StudentId, Name, Age, Department)
VALUES
    (2, 'Divya', 22, 'ECE'),
    (3, 'Kumar', 20, 'CSE');
```

Insert Without Specifying Every Column:

```
None
INSERT INTO Students (StudentId, Name)
VALUES (4, 'Priya');
```

Columns not supplied must allow NULL, have a DEFAULT value, or be otherwise generated by the database.

16. SELECT Statement

The SELECT statement retrieves data from one or more tables.

Retrieve All Columns:

```
None
SELECT * FROM Students;
```

Retrieve Specific Columns:

```
None
SELECT StudentId, Name
FROM Students;
```

Using Column Alias:

```
None
SELECT Name AS StudentName
FROM Students;
```

Using DISTINCT:

DISTINCT removes duplicate rows from the selected result.

```
None
SELECT DISTINCT Department
FROM Students;
```

17. WHERE Clause

The WHERE clause filters records based on specified conditions.

Example:

```
None
SELECT *
FROM Students
WHERE Age > 20;
```

Important Comparison Operators:

- = Equal to
- <> Not equal to
- != Not equal to (also supported in SQL Server)
- Greater than
- < Less than
- = Greater than or equal to
- <= Less than or equal to

Example:

```
None
SELECT *
FROM Students
WHERE Department = 'CSE';
```

18. Logical Operators

Logical operators combine multiple conditions.

AND:

Both conditions must be true.

```
None
SELECT *
FROM Students
WHERE Age > 20 AND Department = 'CSE';
```

OR:

At least one condition must be true.

```
None
SELECT *
FROM Students
WHERE Department = 'CSE'
   OR Department = 'ECE';
```

NOT:

Reverses a condition.

```
None
SELECT *
FROM Students
WHERE NOT Department = 'CSE';
```

19. BETWEEN Operator

BETWEEN checks whether a value is within an inclusive range.

Example:

```
None
SELECT *
FROM Students
WHERE Age BETWEEN 18 AND 25;
```

This includes both 18 and 25.

20. IN Operator

IN checks whether a value matches any value in a specified list.

Example:

```
None
SELECT *
FROM Students
WHERE Department IN ('CSE', 'ECE', 'IT');
```

21. LIKE Operator

LIKE is used for pattern matching.

Common wildcard characters in SQL Server:

- %: Matches zero or more characters.
- _: Matches exactly one character.
- []: Matches any one character in the specified set.

- [^]: Matches a character not in the specified set.

Examples:

Names Starting with A:

```
None
SELECT *
FROM Students
WHERE Name LIKE 'A%';
```

Names Ending with A:

```
None
SELECT *
FROM Students
WHERE Name LIKE '%a';
```

Names Containing iv:

```
None
SELECT *
FROM Students
WHERE Name LIKE '%iv%';
```

Names with Exactly Five Characters:

```
None
SELECT *
FROM Students
WHERE Name LIKE '_____';
```

22. IS NULL and IS NOT NULL

NULL represents missing or unknown data.

Use IS NULL to find rows where a column has a NULL value.

```
None
SELECT *
FROM Students
WHERE Age IS NULL;
```

Use IS NOT NULL to find rows where a column has a value.

```
None
SELECT *
FROM Students
WHERE Age IS NOT NULL;
```

Important:

Do not use `= NULL` or `!= NULL` to test for NULL values. Use IS NULL or IS NOT NULL.

23. ORDER BY Clause

ORDER BY is used to sort query results.

Ascending Order:

```
None
SELECT *
FROM Students
ORDER BY Age ASC;
```

Descending Order:

```
None
SELECT *
FROM Students
ORDER BY Age DESC;
```

Sort Using Multiple Columns:

```
None
SELECT *
FROM Students
ORDER BY Department ASC, Age DESC;
```

- ASC: Ascending order.
- DESC: Descending order.

24. TOP Clause in SQL Server

TOP is used in SQL Server to limit the number of rows returned.

Example:

```
None
SELECT TOP 5 *
FROM Students;
```

TOP with ORDER BY:

```
None
SELECT TOP 3 *
FROM Students
ORDER BY Age DESC;
```

This returns the three students with the highest ages.

25. UPDATE Statement

UPDATE modifies existing records.

Syntax:

```
None
UPDATE TableName
SET ColumnName = Value
WHERE Condition;
```

Example:

```
None
UPDATE Students
SET Age = 23
WHERE StudentId = 1;
```

Update Multiple Columns:

```
None
UPDATE Students
SET Name = 'Arun Kumar',
    Age = 22
WHERE StudentId = 1;
```

Important:

Always check the WHERE condition before running an UPDATE statement. Without WHERE, all rows in the table may be updated.

26. DELETE Statement

DELETE removes selected records from a table.

Example:

```
None
DELETE FROM Students
WHERE StudentId = 3;
```

Delete All Records:

```
None
DELETE FROM Students;
```

The table structure remains.

Important:

Without a WHERE clause, DELETE removes all rows.

27. TRUNCATE TABLE

TRUNCATE TABLE removes all rows from a table while retaining the table structure.

Example:

```
None
TRUNCATE TABLE Students;
```

In SQL Server:

- It removes all rows.
- It does not support a WHERE clause.
- It generally uses fewer individual row-deletion operations than DELETE.
- It resets an identity counter to its seed value.
- It cannot be used when another table references the target table through a foreign key.

28. DROP Statement

DROP removes a database object such as a table or database.

Drop a Table:

```
None
DROP TABLE Students;
```

Drop a Database:

```
None
DROP DATABASE CollegeDB;
```

Use DROP carefully because it removes the specified object.

29. DELETE vs TRUNCATE vs DROP

DELETE:

- Removes selected rows when WHERE is used.
- Can remove all rows without WHERE.

- Retains the table structure.
- Supports row-level triggers in SQL Server.
- Can be rolled back when performed inside an active transaction that is rolled back.

TRUNCATE:

- Removes all rows.
- Does not support WHERE.
- Retains the table structure.
- Resets identity values to their seed.
- Has restrictions involving foreign key references.
- Can be rolled back when performed inside an appropriate SQL Server transaction.

DROP:

- Removes the table or other specified database object.
- Removes the object's structure as well as its data.

30. ALTER TABLE

ALTER TABLE is used to modify an existing table structure.

Add a Column:

```
None
ALTER TABLE Students
ADD Email NVARCHAR(100);
```

Modify a Column Data Type:

```
None
ALTER TABLE Students
ALTER COLUMN Name NVARCHAR(150);
```

Drop a Column:

```
None
ALTER TABLE Students
DROP COLUMN Email;
```

Add a Primary Key:

```
None
ALTER TABLE Students
ADD CONSTRAINT PK_Students
PRIMARY KEY (StudentId);
```

Add a Foreign Key:

```
None
ALTER TABLE Students
ADD CONSTRAINT FK_Students_Departments
FOREIGN KEY (DepartmentId)
REFERENCES Departments(DepartmentId);
```

31. Aggregate Functions

Aggregate functions perform calculations on a set of rows and return a single result.

Important aggregate functions:

COUNT():

Counts rows.

```
None
SELECT COUNT(*)
FROM Students;
```

SUM():

Calculates the total.

```
None
SELECT SUM(Mark)
FROM ExamResults;
```

AVG():

Calculates the average.

```
None
SELECT AVG(Mark)
FROM ExamResults;
```

MAX():

Returns the maximum value.

```
None
SELECT MAX(Mark)
FROM ExamResults;
```

MIN():

Returns the minimum value.

```
None
SELECT MIN(Mark)
FROM ExamResults;
```

Note:

- COUNT(*) counts rows.
- COUNT(ColumnName) counts non-NULL values in that column.
- Most aggregate functions ignore NULL values.

32. GROUP BY Clause

GROUP BY groups records with the same values in specified columns.

Example:

```
None
SELECT Department, COUNT(*) AS TotalStudents
FROM Students
GROUP BY Department;
```

This counts the students in each department.

Another Example:

```
None
SELECT Department, AVG(Age) AS AverageAge
FROM Students
GROUP BY Department;
```

33. HAVING Clause

HAVING filters grouped results.

Example:

```
None
SELECT Department, COUNT(*) AS TotalStudents
FROM Students
GROUP BY Department
HAVING COUNT(*) > 5;
```

This displays departments having more than five students.

Difference Between WHERE and HAVING:

WHERE:

- Filters individual rows before grouping.
- Cannot directly use aggregate conditions in the same query level.

HAVING:

- Filters groups after GROUP BY.
- Commonly used with aggregate functions.

34. SQL Execution Order

A simplified logical processing order for a SELECT query is:

1. FROM
2. JOIN / ON
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. DISTINCT

8. ORDER BY
9. TOP

This helps understand why aliases created in SELECT generally cannot be used in WHERE at the same query level.

35. Joins in SQL

A JOIN combines related data from two or more tables.

Suppose we have:

Students Table:

StudentId	Name	DepartmentId
1	Arun	101
2	Divya	102
3	Kumar	101
4	Priya	NULL

Departments Table:

DepartmentId	DepartmentName
101	CSE
102	ECE
103	IT

36. INNER JOIN

INNER JOIN returns rows that have matching values in both tables.

Example:

```
None
SELECT
    s.StudentId,
    s.Name,
```

```
    d.DepartmentName
FROM Students s
INNER JOIN Departments d
    ON s.DepartmentId = d.DepartmentId;
```

Result:

StudentId	Name	DepartmentName
1	Arun	CSE
2	Divya	ECE
3	Kumar	CSE

Priya is not included because there is no matching DepartmentId.

37. LEFT JOIN

LEFT JOIN returns all rows from the left table and matching rows from the right table.

If there is no matching row, columns from the right table contain NULL.

Example:

```
None
SELECT
    s.StudentId,
    s.Name,
    d.DepartmentName
FROM Students s
LEFT JOIN Departments d
    ON s.DepartmentId = d.DepartmentId;
```

Priya is included with NULL for DepartmentName.

38. RIGHT JOIN

RIGHT JOIN returns all rows from the right table and matching rows from the left table.

Example:

```
None
SELECT
    s.Name,
    d.DepartmentName
FROM Students s
RIGHT JOIN Departments d
    ON s.DepartmentId = d.DepartmentId;
```

All departments are included, even departments that have no students.

39. FULL OUTER JOIN

FULL OUTER JOIN returns:

- Matching records from both tables.
- Unmatched records from the left table.
- Unmatched records from the right table.

Example:

```
None
SELECT
    s.Name,
    d.DepartmentName
FROM Students s
FULL OUTER JOIN Departments d
    ON s.DepartmentId = d.DepartmentId;
```

40. CROSS JOIN

CROSS JOIN returns the Cartesian product of two tables.

Every row from the first table is combined with every row from the second table.

Example:

```
None
SELECT
```

```
    s.Name,
    d.DepartmentName
FROM Students s
CROSS JOIN Departments d;
```

If the first table has 4 rows and the second table has 3 rows, the result contains 12 combinations.

41. SELF JOIN

A SELF JOIN joins a table with itself.

It is useful when records in the same table have a relationship.

Example: Employee and Manager.

```
None
SELECT
    e.EmployeeName,
    m.EmployeeName AS ManagerName
FROM Employees e
LEFT JOIN Employees m
    ON e.ManagerId = m.EmployeeId;
```

Here, the Employees table is used twice with different aliases.

42. Difference Between SQL Joins

- INNER JOIN: Returns matching records only.
- LEFT JOIN: Returns all left-side records and matching right-side records.
- RIGHT JOIN: Returns all right-side records and matching left-side records.
- FULL OUTER JOIN: Returns all records from both sides, matching where possible.
- CROSS JOIN: Returns all possible combinations.
- SELF JOIN: Joins a table with itself.

43. Subqueries

A subquery is a query written inside another SQL query.

It can be used in SELECT, FROM, WHERE and other supported query contexts.

Example:

Find students whose marks are greater than the average mark.

```
None
SELECT *
FROM ExamResults
WHERE Mark >
(
    SELECT AVG(Mark)
    FROM ExamResults
);
```

The inner query calculates the average mark. The outer query retrieves students whose marks exceed it.

44. Types of Subqueries

1. Single-Row Subquery

Returns one row.

```
None
SELECT *
FROM Employees
WHERE Salary =
(
    SELECT MAX(Salary)
    FROM Employees
);
```

2. Multiple-Row Subquery

Returns multiple rows.

```
None
SELECT *
FROM Students
WHERE DepartmentId IN
```

```
(
    SELECT DepartmentId
    FROM Departments
    WHERE DepartmentName IN ('CSE', 'ECE')
);
```

3. Correlated Subquery

A correlated subquery refers to a column from the outer query and is logically evaluated in relation to each outer row.

Example:

```
None
SELECT *
FROM Employees e
WHERE Salary >
(
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentId = e.DepartmentId
);
```

This finds employees whose salaries are greater than their department's average salary.

45. EXISTS and NOT EXISTS

EXISTS checks whether a subquery returns at least one row.

Example:

```
None
SELECT *
FROM Departments d
WHERE EXISTS
(
    SELECT 1
    FROM Students s
```

```
    WHERE s.DepartmentId = d.DepartmentId
);
```

This returns departments that have at least one student.

NOT EXISTS:

Returns rows for which the subquery does not return a matching row.

```
None
SELECT *
FROM Departments d
WHERE NOT EXISTS
(
    SELECT 1
    FROM Students s
    WHERE s.DepartmentId = d.DepartmentId
);
```

This finds departments without students.

46. Set Operators

Set operators combine the results of multiple SELECT statements.

UNION:

Combines results and removes duplicate rows.

```
None
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;
```

UNION ALL:

Combines results without removing duplicate rows.

```
None
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers;
```

INTERSECT:

Returns rows common to both query results.

```
None
SELECT City FROM Customers
INTERSECT
SELECT City FROM Suppliers;
```

EXCEPT:

Returns rows from the first query that are not present in the second query.

```
None
SELECT City FROM Customers
EXCEPT
SELECT City FROM Suppliers;
```

Important:

- Each query must return the same number of columns.
- Corresponding columns must have compatible data types.
- UNION removes duplicates, while UNION ALL retains them.

47. Views

A view is a database object that represents a stored SELECT query.

It presents data from one or more tables as a virtual table.

Example:

```
None
CREATE VIEW StudentDetails AS
```

```
SELECT
    s.StudentId,
    s.Name,
    d.DepartmentName
FROM Students s
INNER JOIN Departments d
    ON s.DepartmentId = d.DepartmentId;
```

Using a view:

```
None
SELECT *
FROM StudentDetails;
```

Advantages:

- Simplifies complex queries.
- Can provide controlled access to selected data.
- Improves query reusability.
- Separates the presentation of data from the underlying table structure.

Important:
A standard view does not necessarily store its own copy of the result data.

48. Stored Procedures

A stored procedure is a named database routine that contains SQL statements and can be executed when needed.

Example:

```
None
CREATE PROCEDURE GetAllStudents
AS
BEGIN
    SELECT *
    FROM Students;
END;
```

Execute:

```
None
EXEC GetAllStudents;
```

Stored Procedure with a Parameter:

```
None
CREATE PROCEDURE GetStudentById
    @StudentId INT
AS
BEGIN
    SELECT *
    FROM Students
    WHERE StudentId = @StudentId;
END;
```

Execute:

```
None
EXEC GetStudentById @StudentId = 1;
```

Advantages:

- Reuses database logic.
- Supports parameters.
- Helps organize complex operations.
- Can help manage permissions.
- Can improve performance in appropriate scenarios.

49. Functions in SQL Server

A function is a database object that returns a value or a table.

SQL Server supports different types of user-defined functions.

1. Scalar Function

Returns a single value.

Example:

```
None
CREATE FUNCTION GetSquare
(
    @Number INT
)
RETURNS INT
AS
BEGIN
    RETURN @Number * @Number;
END;
```

Usage:

```
None
SELECT dbo.GetSquare(5) AS Result;
```

2. Inline Table-Valued Function

Returns a table using a single SELECT statement.

Example:

```
None
CREATE FUNCTION GetStudentsByDepartment
(
    @DepartmentId INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT *
    FROM Students
    WHERE DepartmentId = @DepartmentId
);
```

Usage:

```
None
SELECT *
FROM dbo.GetStudentsByDepartment(101);
```

50. Stored Procedure vs Function

Stored Procedure:

- Executed using EXEC or EXECUTE.
- Can return result sets and output parameters.
- Can perform database modifications.
- Can manage transactions.
- Can return an integer status code.

Function:

- Must return a scalar value or a table.
- Can be used in supported SQL expressions and query contexts.
- Has restrictions on side effects and data modification compared with stored procedures.

51. Triggers

A trigger is a special database routine that automatically executes in response to specified database events.

In SQL Server, DML triggers can execute after or instead of INSERT, UPDATE or DELETE operations.

Example:

```
None
CREATE TRIGGER trg_StudentInsert
ON Students
AFTER INSERT
AS
BEGIN
    PRINT 'Student record inserted';
END;
```

Types of SQL Server Triggers:

1. DML Triggers: Respond to INSERT, UPDATE and DELETE operations.

2. DDL Triggers: Respond to database definition events such as CREATE, ALTER and DROP.
3. LOGON Triggers: Respond to login events.

Important:
In SQL Server, DML triggers are statement-level, not automatically row-by-row. They use the inserted and deleted logical tables to access affected rows.

52. Indexes

An index is a database structure used to help retrieve records efficiently.

It can improve query performance when designed appropriately.

Example:

```
None
CREATE INDEX IX_Students_Name
ON Students(Name);
```

This creates an index on the Name column.

Types of Indexes:

1. Clustered Index

- Determines the logical ordering of data rows in the table according to its index key.
- A table can have only one clustered index.

2. Nonclustered Index

- Stores index keys and row locators separately from the underlying table data.
- A table can have multiple nonclustered indexes.

Advantages:

- Faster data retrieval for suitable queries.
- Helps filtering and sorting.
- Can improve JOIN performance.

Disadvantages:

- Requires additional storage.
- Can slow down INSERT, UPDATE and DELETE operations because indexes may need maintenance.

- Too many indexes can reduce overall performance.

53. Transactions

A transaction is a group of database operations treated as a logical unit of work.

A transaction should either complete successfully or be rolled back when required.

Example:

```
None
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 1000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 1000
WHERE AccountId = 2;

COMMIT TRANSACTION;
```

The transaction transfers 1000 from one account to another.

If an error occurs, the transaction can be rolled back.

```
None
ROLLBACK TRANSACTION;
```

54. ACID Properties

ACID properties ensure reliable transaction processing.

1. Atomicity

A transaction is treated as one logical unit. Its operations either complete together or are rolled back.

2. Consistency

A transaction should preserve database rules and constraints.

3. Isolation

Concurrent transactions should not interfere with each other in ways that violate the selected isolation guarantees.

4. Durability

Once a transaction is committed, its changes are designed to survive failures according to the database's durability guarantees.

55. COMMIT and ROLLBACK

COMMIT:

Permanently commits the changes made by the current transaction.

```
None
BEGIN TRANSACTION;

INSERT INTO Students
    (StudentId, Name)
VALUES
    (10, 'Ravi');

COMMIT TRANSACTION;
```

ROLLBACK:

Cancels the uncommitted changes in the current transaction.

```
None
BEGIN TRANSACTION;

DELETE FROM Students
WHERE StudentId = 10;

ROLLBACK TRANSACTION;
```

56. SAVEPOINT

A savepoint allows a transaction to roll back to a specific point without rolling back the entire transaction.

In SQL Server, SAVE TRANSACTION is used.

Example:

```
None
BEGIN TRANSACTION;

INSERT INTO Students
    (StudentId, Name)
VALUES
    (11, 'Kumar');

SAVE TRANSACTION StudentSave;

UPDATE Students
SET Name = 'Arun'
WHERE StudentId = 11;

ROLLBACK TRANSACTION StudentSave;

COMMIT TRANSACTION;
```

Here, the INSERT is retained and the UPDATE is rolled back.

57. Normalization

Normalization is the process of organizing relational database tables to reduce unnecessary data duplication and improve data integrity.

Why is Normalization Required?

- Reduces redundant data.
- Prevents update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- Improves data organization.

58. First Normal Form (1NF)

A table is in First Normal Form when:

- Each column contains a single value for each row.
- Values are atomic according to the data model.
- Rows can be uniquely identified.

Incorrect design:

StudentId	Name	PhoneNumbers
1	Arun	9876, 8765

The PhoneNumbers column contains multiple values.

Better design:

Students:

StudentId	Name
1	Arun

StudentPhones:

StudentId	PhoneNumber
1	9876
1	8765

59. Second Normal Form (2NF)

A table is in Second Normal Form when:

- It is already in 1NF.
- Every non-key attribute is fully functionally dependent on the entire candidate key.
- There are no partial dependencies on part of a composite candidate key.

Example:

Consider this table:

StudentId	CourseId	StudentName	CourseName

The candidate key is StudentId + CourseId.

StudentName depends only on StudentId.

CourseName depends only on CourseId.

These partial dependencies can be removed by splitting the table.

Students:

StudentId	StudentName

Courses:

CourseId	CourseName

Enrollments:

StudentId	CourseId

60. Third Normal Form (3NF)

A table is in Third Normal Form when:

- It is already in 2NF.
- It has no transitive dependency of non-key attributes on a candidate key.

Example:

Consider:

StudentId	StudentName	DepartmentId	DepartmentName

DepartmentName depends on DepartmentId rather than directly on StudentId.

Split the data into:

Students:

StudentId	StudentName	DepartmentId

Departments:

DepartmentId	DepartmentName

This reduces unnecessary duplication.

61. ER Model and Relationships

ER stands for Entity-Relationship.

An ER model describes entities, their attributes and relationships.

Entity:

A real-world object represented in a database.

Examples:

- Student.
- Employee.
- Department.
- Course.

Attribute:

A property of an entity.

Example:

- StudentId.
- Name.
- Age.

Relationship:

Describes how entities are connected.

62. Types of Relationships

1. One-to-One

One record in a table is related to at most one record in another table, and vice versa.

Example:

- Person and Passport.

2. One-to-Many

One record in a table can be related to multiple records in another table.

Example:

- One Department has many Students.

3. Many-to-Many

Multiple records in one table can be related to multiple records in another table.

Example:

- Students can enroll in multiple Courses.
- Each Course can have multiple Students.

A junction table is used to implement a many-to-many relationship.

Example:

StudentCourses:

StudentId	CourseId
1	101
1	102
2	101

63. Identity Column in SQL Server

IDENTITY automatically generates sequential numeric values for a column.

Example:

```
None
CREATE TABLE Students
(
    StudentId INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Name NVARCHAR(100)
);
```

Here:

- First 1: Seed value.
- Second 1: Increment value.

Insert:

```
None
INSERT INTO Students (Name)
VALUES ('Arun');
```

The StudentId is generated automatically.

64. Retrieve the Last Inserted Identity Value

In SQL Server, SCOPE_IDENTITY() returns the last identity value generated in the same scope.

Example:

```
None
INSERT INTO Students (Name)
VALUES ('Divya');

SELECT SCOPE_IDENTITY();
```

It is commonly used when an application needs to retrieve the newly generated identity value.

65. SQL Server Date and String Functions

Date Functions:

GETDATE():

Returns the current SQL Server system date and time as DATETIME.

```
None
SELECT GETDATE();
```

SYSDATETIME():

Returns the current system date and time with higher precision.

```
None
SELECT SYSDATETIME();
```

YEAR():

```
None
SELECT YEAR(GETDATE());
```

MONTH():

```
None
SELECT MONTH(GETDATE());
```

DAY():

```
None
SELECT DAY(GETDATE());
```

DATEDIFF():

Calculates the difference between two date/time values in a specified unit.

```
None
SELECT DATEDIFF(YEAR, '2000-01-01', GETDATE());
```

String Functions:

UPPER():

```
None
SELECT UPPER('hello');
```

LOWER():

```
None
SELECT LOWER('HELLO');
```

LEN():

```
None
SELECT LEN('Database');
```

Note: LEN in SQL Server excludes trailing spaces.

SUBSTRING():

```
None
SELECT SUBSTRING('DATABASE', 1, 4);
```

REPLACE():

```
None
SELECT REPLACE('Hello World', 'World', 'SQL');
```

CONCAT():

```
None
SELECT CONCAT('Hello', ' ', 'SQL');
```

TRIM():

```
None
SELECT TRIM('  Hello  ');
```

66. CASE Expression

CASE is used to return values based on conditions.

Example:

```
None
SELECT
    Name,
    Mark,
    CASE
        WHEN Mark >= 90 THEN 'A Grade'
        WHEN Mark >= 75 THEN 'B Grade'
        WHEN Mark >= 50 THEN 'C Grade'
        ELSE 'Fail'
    END AS Grade
FROM ExamResults;
```

It is similar to conditional logic in programming languages.

67. Common Table Expressions (CTE)

A Common Table Expression is a named temporary result set defined within a single SQL statement.

Example:

```
None
WITH HighScorers AS
(
    SELECT *
    FROM ExamResults
    WHERE Mark > 80
)
SELECT *
FROM HighScorers;
```

Advantages:

- Improves readability.
- Simplifies complex queries.
- Can be used with recursive queries.

- Helps organize multi-step query logic.

A CTE exists only for the statement that immediately follows its definition.

68. Temporary Tables

Temporary tables store temporary data during a database session or procedure execution.

In SQL Server:

Local Temporary Table:

```
None
CREATE TABLE #TempStudents
(
    StudentId INT,
    Name NVARCHAR(100)
);
```

Insert data:

```
None
INSERT INTO #TempStudents
VALUES (1, 'Arun');
```

Retrieve data:

```
None
SELECT *
FROM #TempStudents;
```

Global Temporary Table:

```
None
CREATE TABLE ##GlobalStudents
(
    StudentId INT,
    Name NVARCHAR(100)
```

```
);
```

Difference:

- #TempTable is local to the session and has session-based visibility.
- ##TempTable can be accessed by other sessions while it exists.

69. SQL Injection

SQL Injection is a security vulnerability that occurs when untrusted input is incorrectly incorporated into SQL commands.

Example of unsafe query construction:

```
None
string query =
    "SELECT * FROM Users WHERE Name = '" + userInput + "'";
```

An attacker may provide specially crafted input that changes the meaning of the SQL statement.

How to Prevent SQL Injection:

1. Use parameterized queries.
2. Use Entity Framework Core safely.
3. Validate user input.
4. Avoid building SQL commands by directly concatenating untrusted strings.
5. Follow the principle of least privilege.

Parameterized SQL Example:

```
None
using Microsoft.Data.SqlClient;

string query =
    "SELECT * FROM Users WHERE Name = @Name";

using SqlCommand command = new SqlCommand(query, connection);

command.Parameters.AddWithValue("@Name", userInput);
```

For better type control, explicitly specify the SQL parameter type and size instead of relying on inferred types.

70. Backup and Restore

A database backup creates a copy of database data that can be used for recovery.

Backup in SQL Server:

```
None
BACKUP DATABASE CollegeDB
TO DISK = 'C:\Backups\CollegeDB.bak'
WITH INIT;
```

Restore Database:

```
None
RESTORE DATABASE CollegeDB
FROM DISK = 'C:\Backups\CollegeDB.bak';
```

Important:

- Backup protects data from loss.
- Restore recovers data from a backup.
- Production databases need a planned backup and recovery strategy.
- Backup files must be stored securely and tested periodically.

71. SQL and Entity Framework Core

Entity Framework Core is an Object-Relational Mapper (ORM).

It allows developers to work with database records using C# objects instead of writing SQL manually for every operation.

Important EF Core Concepts:

DbContext:

Manages database operations, entity tracking and saving changes.

DbSet:

Represents a collection of entities of a particular type.

DbContextOptions:

Contains configuration required by DbContext, such as the database provider and connection details.

Example:

```
None
public class AppDbContext : DbContext
{
    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

Registering DbContext:

```
None
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")
    )
);
```

Saving Data:

```
None
Student student = new Student
{
    Name = "Divya",
    Age = 21
};
```

```
_context.Students.Add(student);

await _context.SaveChangesAsync();
```

72. SQL vs Entity Framework Core

SQL:

- A query language used to communicate with relational databases.
- Provides direct control over SQL queries.
- Used to create, retrieve and modify database data.

Entity Framework Core:

- An ORM for .NET applications.
- Maps C# objects to database structures.
- Generates SQL for supported LINQ and change-tracking operations.
- Provides migrations and database access through C#.

Important:

EF Core does not replace SQL. Understanding SQL helps developers write efficient queries and understand what EF Core generates.